LOGARITHMIC MERGING

Dynamic inverted indexing in C++17

Implementation and experimental report

This project implements a dynamic inverted index that keeps new documents searchable while limiting the cost of rewriting disk indexes. It follows the logarithmic merging policy described in *Introduction to Information Retrieval*, Figure 4.7.

Collection	50,000 documents | 5,678,185 postings | 50,000 terms
Implementation	C++17, provided streaming merge framework
Compiler	g++ 13.3.0 with -O2 -Wall -Wextra -pedantic
Test result	12/12 tests passed on tiny and 50k collections
Measured host	Linux x86_64 | AMD EPYC 9V74 | 9 vCPUs | 9.7 GiB RAM

The repository contains the implementation, provided framework, tests, experiment traces, and this report. The generated 50k-document corpus can be recreated with the included corpus generator.

1. Implementation

The in-memory auxiliary index Z0 accepts documents until the next document would push its posting count over the configured limit n. A flush writes Z0 to a temporary disk index and clears the in-memory structure. That new index is treated as a carry.

Each occupied level acts like a set bit in a binary counter. At level i, the older generation Ii is merged before the newer carry. The merge output becomes the next carry and both input files are removed after their reader-cache entries are forgotten. The first empty level takes the carry. This leaves at most one index at each level and keeps the occupied-level mask equal to the number of flushes in binary.

Lookup and finalization

A lookup starts with the postings in Z0 and takes a sorted union with the postings found in every occupied generation. Therefore a query sees all documents already added, including documents whose most recent batch has not been flushed. The memory index and every disk level hold disjoint document ranges, so their union is complete and ordered.

Finalization flushes any remaining Z0 postings. With no generations, it writes an empty index; with one generation, it renames that index; with several, it calls the provided k-way merge once with generations ordered from older to newer. Using the streaming helper avoids loading full indexes into memory. Temporary generations are removed after the final merge, and the destructor removes any generations left when a caller does not finalize.

Implementation checks

The source file uses the project APIs for all disk reads and writes. Each merge increments the merge counter. ReaderCache entries are forgotten before any backing index is deleted or renamed. The implementation compiled with the requested warning flags, and the supplied suite passed on both the 12-document debugging corpus and the generated 50,000-document corpus.

> The custom implementation is intentionally direct: a loop models the binary carry, a vector stores generation paths, and helper functions handle posting unions and disk merges.

2. Experimental method and E1 results

The provided generator created the deterministic Zipf-like corpus with seed 42. Experiments ran in Release-style builds compiled with g++ -O2. For each auxiliary size n, both the auxiliary-plus-main baseline and logarithmic merging indexed the same collection. The trace records one row after each flush. Final index correctness was checked with the provided verifier in every run.

E1: total merge work by auxiliary size

Policy	n	Flushes	Merges	Peak disk indexes	Postings written*	Including finalize	Amplification*
Aux + main	50000	113	112	1	327.3M	333.0M	57.64x
Logarithmic	50000	113	109	6	36.0M	41.8M	6.34x
Aux + main	100000	56	55	1	165.0M	170.8M	29.06x
Logarithmic	100000	56	53	5	30.4M	36.1M	5.35x
Aux + main	200000	28	27	1	86.6M	92.3M	15.25x
Logarithmic	200000	28	25	4	24.8M	30.6M	4.37x
Aux + main	400000	14	13	1	47.2M	52.9M	8.31x
Logarithmic	400000	14	11	3	19.2M	25.0M	3.38x

* Postings written and write amplification are measured before finalize. “Including finalize” reports the total after the final index is created.

Across the tested n values, logarithmic merging wrote 2.46x to 9.09x fewer postings during indexing than the baseline. Including finalization, its total writes were 2.12x to 7.97x lower. The relative gain is largest at smaller n, where the baseline repeatedly rewrites a growing main index many more times.

At n = 100,000, the baseline wrote 165.0 million postings during indexing, compared with 30.4 million for logarithmic merging. The logarithmic index used at most five disk generations in this run. The difference in wall time followed the same direction, but time is more sensitive to the host and filesystem than the posting write count.

3. E2 trace figures

The plots use the n = 100,000 traces. The horizontal axis is the number of postings indexed so far. The first figure compares cumulative postings written before finalization; the second shows how many logarithmic generations are occupied after each flush.

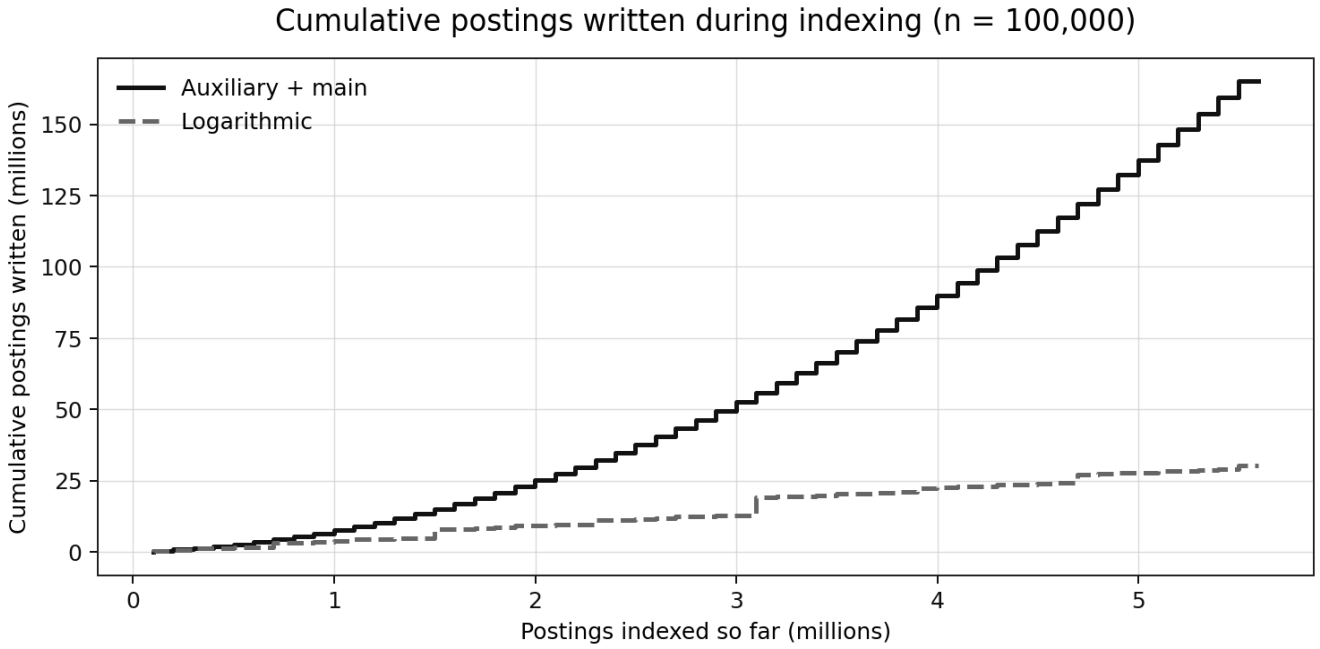


Figure 1. The auxiliary-plus-main policy rewrites its growing main index at each flush. The logarithmic policy has larger jumps during carries, but each posting participates in only a small number of merges.

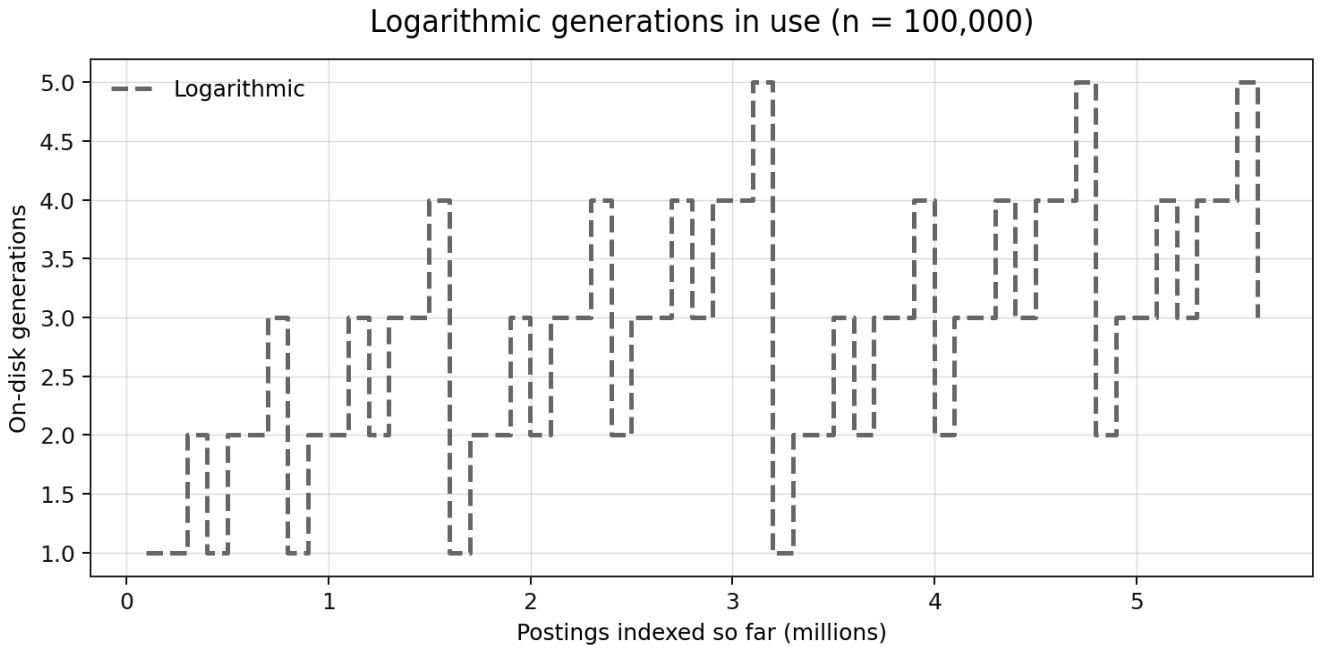


Figure 2. Generation count changes according to the set bits of the flush counter. The count can fall after a carry merges multiple occupied levels.

4. Analysis and conclusions

Merge-cost growth

With an auxiliary limit n and T postings, the baseline repeatedly merges each flushed batch into one growing index. Its total write work grows on the order of T^2/n. Logarithmic merging stores runs in levels of doubling size, so a posting is rewritten at most once per occupied level. Its merge work grows on the order of T log■(T/n). The measurements show this expected pattern: logarithmic merging has much lower write amplification, especially when n is small.

Search and space trade-off

The baseline has at most one disk index, while logarithmic merging keeps several generations and checks each one for a term. The generation count is bounded by the number of bits needed to represent the flush count, or roughly log■(T/n). The experiment observed a maximum of six generations at n = 50,000 and three at n = 400,000. This shifts some work from indexing to lookup, which is the central trade-off of dynamic indexing.

Final merge cost

The final index is made with one k-way merge over all non-empty generations, oldest first. Repeated two-input final merges would rewrite older postings once per merge step. The experiment output reports the additional writes caused by finalization: for n = 100,000, this adds about 5.76 million postings for either policy. This is a one-time cost and leaves a single compact index for later use.

Why observed costs differ from a simple prediction

The asymptotic formulas describe growth, not exact constants. Flushes happen at document boundaries, so actual batch sizes vary. Terms also have uneven document frequencies, and variable-byte encoding changes byte traffic even when the posting count is the same. The framework counts postings, while elapsed time includes dictionary work, filesystem operations, and verification overhead. These details explain why measured values need not match a simplified formula exactly.

Conclusion

The implementation passed all 12 tests on both corpora and produced indexes accepted by the verifier. For this 5.68-million-posting collection, logarithmic merging reduced measured postings written across all tested auxiliary sizes while preserving search correctness during ingestion. Its cost is maintaining several disk indexes and combining their results at query time.

References

Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*, Sections 4.5 and 5.3.1. Cambridge University Press.

Provided assignment starter code, experiment script, verifier, and test suite. C++17 language and standard library documentation.